2024A7PS0002G
AI Lab4: Logic

1. Planning problem:

   Initial state I: I ={At(Robot,A),At(Package,A)}.
   Goal G: G={At(Package,C)}.

   List of actions along with Preconditions and effects:
   i. Move(X,Y)
   At(Robot,X)
   Remove At(Robot,X); add At(Robot,Y)

   ii. PickUp(Package,X)
   At(Robot,X) and At(Package,X)
   Remove At(Package,X); add Holding(Package)

   iii. Drop(Package,X)
   At(Robot,X) and Holding(Package)
   Remove Holding(Package); add At(Package,X)

2.

State	Action Taken	State Facts
S0	Initial State	At(Robot,A), At(Package,A)
S1	PickUp(Package,A)	At(Robot,A), Holding(Package)
S2	Move(A,B)	At(Robot,B), Holding(Package)
S3	Move(B,C)	At(Robot,C), Holding(Package)
S4	Drop(Package,C)	At(Robot,C), At(Package,C)

3. I want to implement a simple planning agent in Python. Represent a state as a set of logical propositions. Each action should contain: - a name; - positive preconditions; - negative preconditions; - positive effects; - negative effects. An action is applicable if all of its preconditions are satisfied by the current state. When an action is applied: 1. remove its negative effects from the state; 2. add its positive effects to the state. Use breadth-first search to find a sequence of actions that achieves a specified goal. The program should also: - detect when no plan exists; - print the resulting sequence of actions; - print the states reached after each action. Explain the implementation and identify any assumptions you make. Run the generated program on the warehouse problem.
   (also provided both above sections)

4. In py file.

5.

```
Plan found:

S0: ['At(Package,A)', 'At(Robot,A)']
1. PickUp(Package,A)
S1: ['At(Robot,A)', 'Holding(Package)']
2. Move(A,B)
S2: ['At(Robot,B)', 'Holding(Package)']
3. Move(B,C)
S3: ['At(Robot,C)', 'Holding(Package)']
4. Drop(Package,C)
S4: ['At(Package,C)', 'At(Robot,C)']
```

6. (Reflection questions)
   6.1) An LLM can only write appropriate code if the problem statement and the approach the user wishes to use are explicitly mentioned and all the constraints and assumptions are conveyed to the LLM.

   6.2) A possible error is the planner carrying out an action whose preconditions are not satisfied. This is a bug.

   6.3) There may be errors in this plan like preconditions being incomplete.

   6.4) The LLM helped generate the Python implementation of the planner, including the action representation, precondition checking, state updates, BFS search, and output formatting.

   6.5) I had to verify that the code worked and that all testcases passed. I also had to verify that the assumptions made by the LLM were accurate.

   6.6) Logical reasoning is used when checking whether an action's preconditions are satisfied in the current state and when determining the resulting state after an action.

   6.7) Search is like finding the ideal path while planning and logic determine whether each step/action taken on that path is actually allowed.

7. LLMs are very useful to generate code and improve prompts even before the code generation. We should plan carefully with LLMs and make sure the goals, methods and expected output is predetermined before we even ask an LLM to generate code.

   All outputs by an LLM must be scrutinised and tested before accepting, as the generated program should not simply be assumed to be correct.